

IE 306 — Systems Simulation

Assignment 3: Drone Light Show Depot — Output Analysis (Group Assignment — Teams of 3)

Spring 2026 | Due Date: Monday, June 1st, 2026, 23:59

Overview

The *Bosphorus Drone Light Festival* stages a nightly aerial choreography performed by a fleet of **200** autonomous drones over the Bosphorus straits. During rehearsal blocks the drones cycle continuously between *flying* a programmed routine and a ground *depot* where each returning drone is battery-swapped and airworthiness-tested before being released back into the show.

The festival's operations manager is considering a capital investment: purchasing one additional swap station for the depot. Your job, as the festival's analytics team, is to **produce a defensible recommendation** using the output-analysis toolkit from Lectures 10 and 11.

The simulator is provided. You will not write a SimPy model in this assignment. The repository ships with `config.py`, `model.py`, and `seeds.py` sitting alongside the notebook — start Jupyter from the repo root and `import config / import model` works directly. You spend your effort on the *analysis*: classification, warmup detection, confidence intervals, paired comparison under Common Random Numbers, and a short verification step. **Do not move, rename, or edit** `config.py`, `model.py`, or `seeds.py`; the autograder relies on them being unchanged.

1 System Description

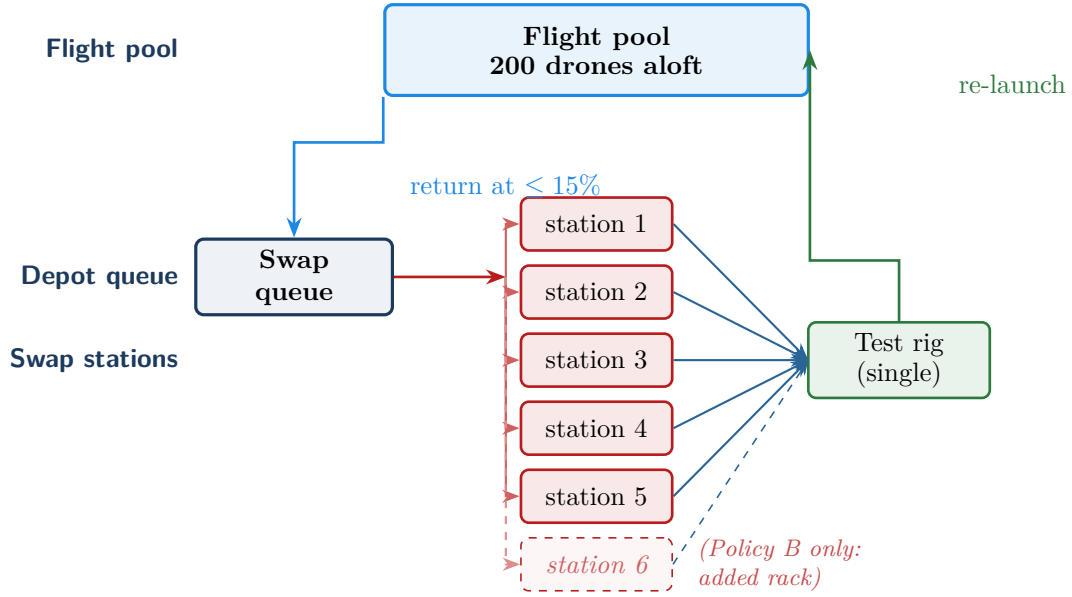


Figure 1: Drone depot system. A closed-loop fleet of 200 drones cycles between the flight pool and the depot. Returns join the swap queue, are served by one of N_{swap} stations, then pass through a single shared test rig before being re-launched. *Policy B* adds a sixth swap station (dashed).

1.1 Fleet and Flight Cycle

A fixed population of $N = 200$ drones operates in a closed loop. Each drone independently follows the cycle

$$\text{fly} \longrightarrow \text{land at depot} \longrightarrow \text{swap} \longrightarrow \text{test} \longrightarrow \text{fly}.$$

A drone returns to the depot when its battery charge falls to or below 15%. While airborne each drone consumes power according to a **lognormal** drain multiplier (mean 1.0, CV 0.18) that scales the nominal full-battery flight time of $T_{\text{full}} = 2400$ s (40 min).

1.2 Depot Service

The depot houses N_{swap} identical swap stations (FCFS, one drone per station at a time) and *one* shared test rig. Service times are **lognormal**:

- **Swap time:** mean $\mu_{\text{swap}} = 45$ s, CV 0.30.
- **Test time:** mean $\mu_{\text{test}} = 8$ s, CV 0.25.

After test, the drone re-launches with a fully charged pack.

1.3 Configurations to Compare

Table 1: The two depot configurations.

Policy	Swap stations	Test rigs
A — Current depot	5	1
B — One extra swap station	6	1

The single test rig is the same in both policies. Service-time distributions, drain distributions, fleet size, and initial conditions are identical. Pairing across the two policies is performed via Common Random Numbers (the provided `seeds.make_streams` returns identical streams when called with the same `(master_seed, rep_index)`).

2 Your Tasks

The notebook `assignment_03_starter.ipynb` ships with one markdown stub and one starter code cell per task. Complete all five.

Task 1. Classification & justification (10 pts). Is the relevant analysis of the depot *terminating* or *steady-state*? Defend your choice in 4–6 sentences. Either is defensible; the remainder of your analysis must be consistent with the choice.

Task 2. Warmup detection (20 pts). Apply *both* Welch’s method (graphical, $R \geq 5$ replications) and MSER (algorithmic, one long run) to the per-drone swap-queue wait series. Report `warmup_welch`, `warmup_mser`, and the value `warmup_chosen` you adopt downstream, with a one-line justification.

Task 3. Two confidence intervals (30 pts). Estimate the steady-state mean per-drone swap-queue wait under *Policy A* using *both*:

- (a) Replications-with-deletion: R independent replications, delete the warmup from each, build a t -CI from the R replication means.
- (b) Batch means on a single long run: after warmup deletion, split the per-completion series into K batches of size B , with B chosen so that $|\hat{\rho}_1(\bar{X}_b)| \leq 0.1$.

Sanity-check that the two CIs overlap. Pick R (for (a)) or B, K (for (b)) so that the 95% half-width is at most 5% of the point estimate.

Task 4. CRN paired comparison (30 pts). Compare Policy A and Policy B using paired replications under Common Random Numbers. Report `task4_paired_diff`, `task4_paired_halfwidth`, and the variance-reduction factor

$$\widehat{\text{VRF}} = \frac{\widehat{\text{Var}}(\bar{X}_A) + \widehat{\text{Var}}(\bar{X}_B)}{\widehat{\text{Var}}(D)}, \quad D_r = \bar{X}_{A,r} - \bar{X}_{B,r}.$$

Conclude with a decision recommendation in `decision_memo.pdf` (one page, addressed to the operations manager).

Task 5. V&V mini (10 pts). Two short checks:

- **V1.** Verify the conservation invariant $\text{FLEET_SIZE} = (\text{in air}) + (\text{in depot})$ holds for the entire run (`state.max_invariant_violation == 0`).
- **V2.** Verify **Little’s Law** on the depot subsystem in steady state: $L_{\text{depot}} \approx \lambda W_{\text{depot}}$ where $L_{\text{depot}} = N - (\text{in air})$, λ is the throughput, and W_{depot} is the mean per-drone sojourn through swap + test. Report the ratio of the two sides; it should be very close to 1.

3 Deliverables

Submit via your team’s GitHub Classroom repository. The repository must contain, at the root:

1. `assignment_03_starter.ipynb` — your completed notebook. It must run end-to-end on a fresh kernel (Restart → Run All) in under 10 minutes.

2. `submission.json` — written automatically by the final cell of the notebook. *Do not edit this file by hand.*
3. `decision_memo.pdf` — one-page memo to the operations manager. Plain prose, no tables. Title, recommendation, evidence (your CI numbers), caveats. PDF, A4.

Do not commit: large data dumps, virtual environments, or any intermediate files. The provided Python modules (`config.py`, `model.py`, `seeds.py`) are read-only — if any of them differ from what was distributed, the autograder will refuse to grade your submission.

4 Grading

70 points are awarded by an autograder that reads the numeric fields in your `submission.json` and scores each against a tolerance band derived from ≥ 200 instructor reference runs. The remaining **30 points** are awarded by the instructor reading your notebook prose, decision memo, and V&V interpretation.

Table 2: Point breakdown.

Component	Auto	Narrative
Task 1 — Classification & justification	4	6
Task 2 — Warmup detection	16	4
Task 3 — Two confidence intervals	24	6
Task 4 — CRN paired comparison + recommendation	18	12
Task 5 — V&V mini	8	2
Total	70	30

The autograder is not distributed; you cannot run it locally. Self-check by running the simulator multiple times and inspecting whether your CIs overlap and shrink with R as expected.

Parameter Summary

Table 3: All numerical parameters (also defined in `solution/config.py`).

Parameter	Symbol	Value
Fleet size	N	200
Full-battery flight time	T_{full}	2400 s (40 min)
Return threshold (battery)		0.15
Drain multiplier	lognormal	mean 1.0, CV 0.18
Swap stations (Policy A / B)	c	5 / 6
Test rigs		1
Swap service time	lognormal	mean 45 s, CV 0.30
Test service time	lognormal	mean 8 s, CV 0.25
Master seed (canonical)		20260601

Show us a clean piece of output analysis. Good luck!